LinkedIn Agentic AI Platform
API Design Document

Course: DATA 236 — Distributed Systems
Professor: Simon Shim
Team:
Akashkumar Senthilkumar – 019162494
Danish Waseem – 019101511
Shruthi Thirukumaran – 019138366
Denisha Ketan Tank – 019138834
Rohil Utture – 018362708
Pramod Satya Dindukurthi – 019124612
Siva Surya Chandran – 019130215
Sadaf Fatima Syeda – 018452733

Base URL: `http://localhost:8000` `(for now)`
Live Docs: `/docs` (Swagger) | `/redoc`

Architecture Overview

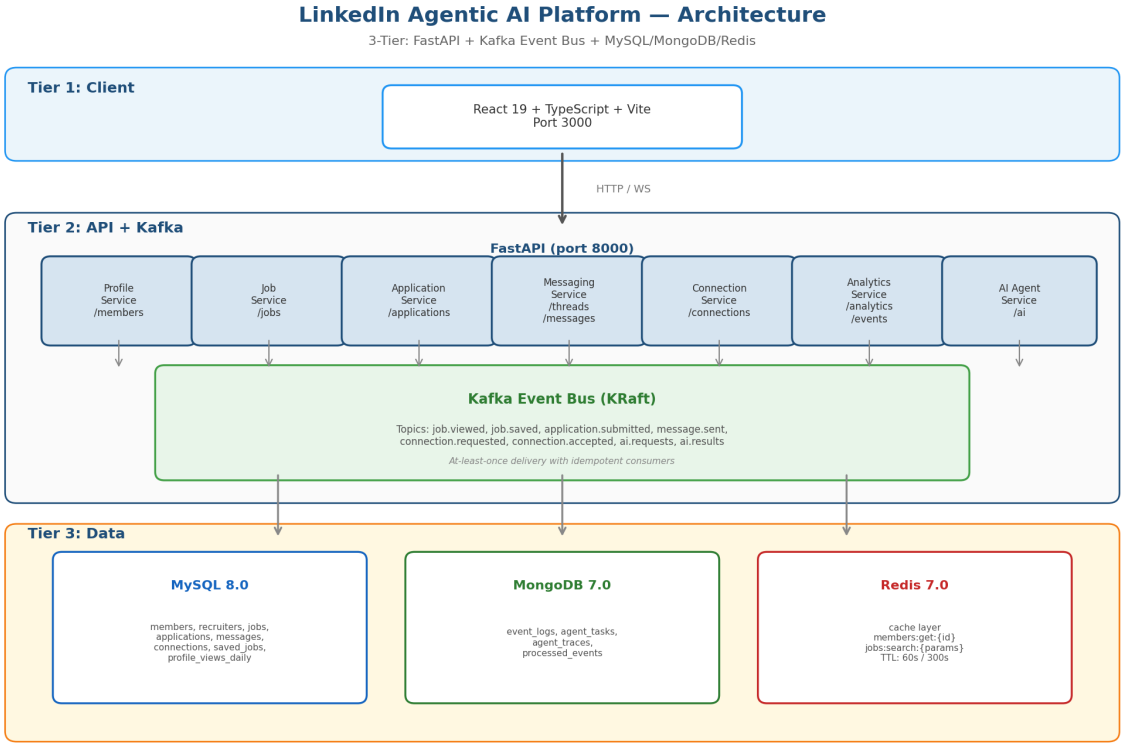

Table of Contents

1. Title and Overview
2. Architecture Summary
3. API Design Principles
4. Service-by-Service API Specification
   - 4.1 Profile Service (Members)
   - 4.2 Recruiter Service
   - 4.3 Job Service
   - 4.4 Application Service
   - 4.5 Messaging Service
   - 4.6 Connection Service
   - 4.7 Analytics Service
   - 4.8 AI Agent Service
   - 4.9 Authentication Service
   - 4.10 System / Health
5. Kafka / Event Integration
6. Database Interaction Summary
7. AI API Workflow
8. API Readiness Summary

1. Overview

System Description

We built a LinkedIn-style professional networking and recruiting platform as a distributed system. The platform covers the full hiring lifecycle: member profiles, job postings, applications, threaded messaging, connection graphs, event-driven analytics, and an AI-powered recruiting copilot that parses resumes, scores candidates, generates outreach drafts, and routes recruiter decisions through a human-in-the-loop approval gate.

Purpose of This Document

This document describes the API surface we designed for the platform: service endpoints, request/response contracts, Kafka event integrations, database interactions, caching behavior, failure handling, and the AI workflow lifecycle. It serves as the API deliverable for the DATA236 group project.

> **Source of truth:** This document reflects the implemented design as of submission. Where any discrepancy exists between this document and the codebase, the code takes precedence. All endpoint behavior can be verified interactively via Swagger UI at `http://localhost:8000/docs`.

Technology Stack

Layer	Technology
API Framework	FastAPI 0.115 (Python 3.11) — async REST + WebSocket
Schema Validation	Pydantic v2 — request/response types and OpenAPI generation
Primary Database	MySQL 8.0 — transactional relational data
Document Store	MongoDB 7.0 — event logs, AI traces, idempotency records
Cache	Redis 7.0 — query result caching, profile lookups
Message Bus	Apache Kafka 3.7 (KRaft, no Zookeeper) — async domain events
AI Inference	Ollama (llama3.2 local LLM) — resume parsing and outreach generation
ORM	SQLAlchemy 2.x — MySQL access with connection pooling
Async Mongo Driver	Motor 3.6 — non-blocking MongoDB operations
Async Kafka Client	aiokafka 0.11 — producer and consumer

2. Architecture Summary

We structured the backend as a FastAPI monolith with clean domain boundaries. Each domain has its own router, schema module, and model module:

Domain	Router prefix	Module
Profile	`/members`	`routers/members.py`
Recruiter	`/recruiters`	`routers/recruiters.py`
Jobs	`/jobs`	`routers/jobs.py`
Applications	`/applications`	`routers/applications.py`
Messaging	`/threads`, `/messages`	`routers/messages.py`
Connections	`/connections`	`routers/connections.py`
Analytics	`/analytics`, `/events`	`routers/analytics.py`
AI Agent	`/ai`	`routers/ai_service.py`

Where Kafka Fits

We positioned Kafka between the REST API layer and the background consumer. REST endpoints act as producers: when a significant domain event occurs (job viewed, application submitted, message sent), the handler publishes a structured JSON event to the appropriate topic and returns immediately. A background consumer (`kafka_consumer.py`) processes these events asynchronously — updating counters, writing to MongoDB, maintaining analytics state.

Where MySQL, MongoDB, and Redis Fit

Store	Holds	Why
MySQL	Members, recruiters, jobs, applications, messages, connections, saved jobs, profile view records	Relational, ACID transactions required, FK constraints for data integrity
MongoDB	Event logs, AI agent task documents, agent traces, Kafka idempotency records	Schema-flexible, write-heavy, unstructured payloads, no JOIN requirements
Redis	Search result caches, individual profile caches	Sub-millisecond lookup, auto-expiry via TTL, no durability needed

How the AI Service Fits

---

3. API Design Principles

Request and Response Format

All API endpoints exchange JSON. We designed every response to follow a consistent envelope (the `Content-Type` header must be `application/json` on requests):

```
{
  "success": true | false,
  "message": "Human-readable status description",
  "data": { ... } | [ ... ] | null
}
```

List responses extend this with pagination fields:

```
{
  "success": true,
  "message": "Found 120 applications",
  "data": [ ... ],
  "total": 120,
  "page": 1,
  "page_size": 20
}
```

Why POST Is Used Across Most Endpoints

1. **Consistent message body** — every operation, including reads, passes its filter parameters in the request body rather than query strings or path segments. This keeps the API uniform and avoids URL length limits for complex filters.
2. **Uniformity** — the API client (frontend, Postman, service-to-service callers) uses one pattern for all calls.
3. **Explicit contract** — Pydantic schemas define exactly what is required and optional for every operation.

Validation Through Pydantic Schemas

We defined every request body as a Pydantic `BaseModel`. Validation runs automatically before the handler executes:

- Required fields declared with `Field(...)` raise a 422 Unprocessable Entity if missing.
- Optional fields default to `None` and are excluded from updates via `model_dump(exclude_unset=True)`.
- String lengths, numeric ranges (`ge=`, `le=`), and enum-style validation are declared at the schema level.
- Swagger UI auto-generates from these schemas, including example payloads.

Error Handling Style

- Genuine server errors (database connection failure, unhandled exception) produce a FastAPI 500 with an `Internal Server Error` body.

Success/Error Response Patterns

Situation	`success`	`message` example	`data`
Normal operation	`true`	`"Member created successfully"`	Entity or list
Cache hit	`true`	`"Job retrieved (cached)"`	Entity
Not found	`false`	`"Member 99 not found"`	`null`
Duplicate	`false`	`"Email 'x@y.com' already exists"`	`null`
Validation failure	`false`	`"Invalid status 'xyz'. Must be one of: ..."`	`null`
Server error	`false`	`"Event ingest failed: ..."`	`null`

Idempotency and Async/Event-Driven Considerations

- We designed the AI task system to be idempotent at the task level: `start_task()` writes to MongoDB before launching the async coroutine, so any crash leaves a queryable record.

4. Service-by-Service API Specification

4.1 Profile Service — `/members`

Base URL prefix: `/members`

`POST /members/create`

Purpose: Create a new member profile.

Request Body:

Field	Type	Required	Description
`first_name`	string	Yes	Min 1, max 100 chars
`last_name`	string	Yes	Min 1, max 100 chars
`email`	string	Yes	Must be unique across all members
`phone`	string	No	
`location_city`	string	No	
`location_state`	string	No	
`location_country`	string	No	
`headline`	string	No	Max 500 chars
`about`	string	No	Free text summary
`experience`	list[object]	No	`[{title, company, years}]`
`education`	list[object]	No	`[{degree, school, year}]`
`skills`	list[string]	No	
`profile_photo_url`	string	No	
`resume_text`	string	No	Extracted resume content

Response: `MemberResponse` — includes full member object with generated `member_id`.

Validation rules: Duplicate email returns `success: false` without inserting.

Cache side effect: Invalidates `members:search:*` pattern.

Databases: MySQL `members` table (INSERT).

POST /members/get

Purpose: Retrieve a member's full profile by ID.
Request Body: `{ "member_id": integer }`
Response: `MemberResponse` — full member profile including all fields.
Cache behavior: Checks `members:get:{member_id}` (TTL 300s). Writes to cache on miss.
Databases: Redis (read), MySQL `members` (read on miss).

POST /members/update

Purpose: Update specific fields on a member profile. Only fields present in the request body are modified.
Request Body: `{ "member_id": integer, <any optional field from MemberCreate> }`
Response: `MemberResponse` — updated member object.
Cache side effect: Deletes `members:get:{member_id}` and `members:search:*`.
Databases: MySQL `members` (UPDATE), Redis (delete keys).

POST /members/delete

Purpose: Permanently delete a member profile and all cascaded records.
Request Body: `{ "member_id": integer }`
Response: `MemberResponse` with `data: null`.
Cache side effect: Deletes `members:get:{member_id}` and `members:search:*`.
Databases: MySQL (DELETE CASCADE — removes applications, connections, saved jobs via FK).

POST /members/search

Purpose: Search members by keyword, skill, or location with cursor-based pagination. All filters are optional and combinable.
Request Body:

Field	Type	Default	Description
`keyword`	string	null	Matches first_name, last_name, headline, about via FULLTEXT (`MATCH…AGAINST`). Falls back to `LIKE` for terms shorter than 3 characters.
`skill`	string	null	Matches within JSON `skills` array
`location`	string	null	Matches location_city or location_state
`sort_by`	string	`"id"`	`"id"` (default, keyset cursor), `"connections"` (offset cursor), `"recent"` (offset cursor)
`page_size`	int	20	Max 100
`cursor`	string	null	Opaque base64 token from the previous response's `next_cursor`; omit for first page

Response: `MemberListResponse`

```
{
  "success": true,
  "data": [...],
  "total": 120,
  "next_cursor": "eyJ0eXBlIjoib2Zmc2V0Iiw...",
  "has_more": true
}
```

Cursor behavior: When `sort_by=id`, the server uses a keyset cursor (`WHERE member_id > last_id`) for stable, index-efficient pagination. For computed sorts (`connections`, `recent`), an offset-encoded cursor is used. Both types are transparent to the caller — the same `cursor` field is used.
Cache behavior: Caches result keyed on all filter params (TTL 60s). Invalidated on any write.
Databases: Redis (read), MySQL `members` (read on miss via FULLTEXT/LIKE queries).

4.2 Recruiter Service — `/recruiters`

Base URL prefix: `/recruiters`

POST /recruiters/create

Purpose: Create a recruiter / employer-admin account.

Request Body:

Field	Type	Required
`first_name`, `last_name`	string	Yes
`email`	string	Yes — unique
`phone`	string	No
`company_id`	int	No
`company_name`	string	No
`company_industry`	string	No
`company_size`	string	No
`role`	string	No — default `"recruiter"`
`access_level`	string	No — default `"standard"`

Validation rules: Duplicate email returns `success: false`.

Databases: MySQL `recruiters`.

POST /recruiters/get

Purpose: Retrieve a recruiter profile by ID.

Request Body: `{ "recruiter_id": integer }`

Cache behavior: `recruiters:get:{recruiter_id}` (TTL 300s).

Databases: Redis, MySQL `recruiters`.

POST /recruiters/update

Purpose: Update specific recruiter fields.

Request Body: `{ "recruiter_id": integer, <any optional recruiter field> }`

Cache side effect: Deletes `recruiters:get:{recruiter_id}`.

Databases: MySQL `recruiters`, Redis.

POST /recruiters/delete

Purpose: Permanently delete a recruiter account.

Request Body: `{ "recruiter_id": integer }`

Cache side effect: Deletes `recruiters:get:{recruiter_id}`.

Databases: MySQL (DELETE CASCADE — removes associated job postings via FK).

4.3 Job Service — `/jobs`

Base URL prefix: `/jobs`

POST /jobs/create

Purpose: Create a new job posting.

Request Body:

Field	Type	Required	Description
`recruiter_id`	int	Yes	Must reference existing recruiter
`title`	string	Yes	
`description`	string	No	
`company_id`	int	No	Defaults to recruiter's company_id
`seniority_level`	string	No	e.g., Entry, Mid, Senior, Director
`employment_type`	string	No	e.g., Full-time, Part-time, Contract

Field	Type	Required	Description
`location`	string	No	
`work_mode`	string	No	`remote`, `hybrid`, `onsite` (default `onsite`)
`skills_required`	list[string]	No	
`salary_min`, `salary_max`	float	No	Annual salary range

Response: `JobResponse` — full job object with generated `job_id`.
Kafka event: `job.created` → topic `job.created` with `{title, location}` payload.
Cache side effect: Invalidates `jobs:search:*`.
Databases: MySQL `job_postings` (INSERT), Kafka.

POST /jobs/get

Purpose: Retrieve full job details by ID. Triggers a view event.
Request Body: `{ "job_id": integer }`
Response: `JobResponse` — full job object.
Kafka event: `job.viewed` → topic `job.viewed`. Consumed by `handle_job_viewed` which increments `views_count` in MySQL.
Cache behavior: `jobs:get:{job_id}` (TTL 300s).
Databases: Redis (read), MySQL `job_postings` (read on miss).

POST /jobs/update

Purpose: Update specific fields of a job posting.
Request Body: `{ "job_id": integer, <any optional job field> }`
Cache side effect: Deletes `jobs:get:{job_id}` and `jobs:search:*`.
Databases: MySQL `job_postings`, Redis.

POST /jobs/search

Purpose: Search open job postings with optional filters and cursor-based pagination. Returns only `status = 'open'` jobs.
Request Body:

Field	Type	Default	Description
`keyword`	string	null	FULLTEXT `MATCH…AGAINST` on title and description. Falls back to `LIKE` for short terms.
`location`	string	null	Partial match on location string
`employment_type`	string	null	Exact match
`work_mode`	string	null	`remote`, `hybrid`, `onsite`
`seniority_level`	string	null	Exact match
`skills`	list[string]	null	All listed skills must be present
`salary_min`	float	null	Minimum salary filter (`salary_max >= salary_min`)
`salary_max`	float	null	Maximum salary filter (`salary_min <= salary_max`)
`sort_by`	string	`"date"`	`"date"` (keyset cursor on `posted_datetime, job_id`), `"applicants"` (offset cursor), `"views"` (offset cursor)
`page_size`	int	20	Max 100
`cursor`	string	null	Opaque base64 token from the previous response's `next_cursor`; omit for first page

Response: `JobListResponse`

```
{
  "success": true,
  "data": [...],
  "total": 47,
  "next_cursor": "eyJ0eXBlIjoia2V5c2V0Iiw...",
  "has_more": true
}
```

Cursor behavior: When `sort_by=date`, a true keyset cursor (`WHERE (posted_datetime, job_id) < (cursor_dt, cursor_id)`) is used — results are stable even as new jobs are inserted. For relevance and computed sorts, an offset-encoded cursor is used. Both are transparent to the caller.

Cache behavior: Keyed on all filter params (TTL 60s).

Databases: Redis, MySQL `job_postings`.

`POST /jobs/close`

Purpose: Close an open job posting. Applications to closed jobs are blocked.

Request Body: `{ "job_id": integer }`

Validation rules: Already-closed job returns `success: false`.

Kafka event: `job.closed`.

Cache side effect: Deletes `jobs:get:{job_id}` and `jobs:search:*`.

Databases: MySQL `job_postings` (status → `"closed"`), Redis.

`POST /jobs/byRecruiter`

Purpose: List all job postings by a specific recruiter.

Request Body: `{ "recruiter_id": integer, "page": 1, "page_size": 20 }`

Response: `JobListResponse` — all statuses included, ordered by `posted_datetime` descending.

Databases: MySQL `job_postings`.

`POST /jobs/save`

Purpose: Save a job posting to a member's saved list.

Request Body: `{ "member_id": integer, "job_id": integer }`

Validation rules: Duplicate save (same member + job) returns `success: false`.

Kafka event: `job.saved`.

Databases: MySQL `saved_jobs` (INSERT), Kafka.

4.4 Application Service — `/applications`

Base URL prefix: `/applications`

Valid status values: `submitted`, `reviewing`, `rejected`, `interview`, `offer`

`POST /applications/submit`

Purpose: Submit a job application.

Request Body:

Field	Type	Required	Description
`job_id`	int	Yes	
`member_id`	int	Yes	
`resume_url`	string	No	URL reference to resume
`resume_text`	string	No	Inline text (falls back to member's stored resume)
`cover_letter`	string	No	
`answers`	object	No	Answers to custom screening questions

Response: `ApplicationResponse` — application object with `application_id`, status `"submitted"`.

Validation rules: - Job must exist and be `open` — else `success: false`. - Member must exist — else `success: false`. - Duplicate (same `job_id` + `member_id`) — returns `success: false`.

Side effects: Increments `job_postings.applicants_count` within the same transaction.

Kafka event: `application.submitted` → topic `application.submitted`, payload: `{job_id, member_id, resume_ref}`. Consumed by `handle_application_submitted` which re-increments `applicants_count` for the Kafka path.

Databases: MySQL `applications` (INSERT), MySQL `job_postings` (UPDATE applicants_count), Kafka.

`POST /applications/get`

Purpose: Retrieve an application by ID.

Request Body: `{ "application_id": integer }`

Databases: MySQL `applications`.

`POST /applications/byJob`

Purpose: List all applications for a job posting (recruiter view).

Request Body: `{ "job_id": integer, "page": 1, "page_size": 20 }`

Response: `ApplicationListResponse` — ordered by `application_datetime` descending.

Databases: MySQL `applications`.

`POST /applications/byMember`

Purpose: List all applications submitted by a member (member view).

Request Body: `{ "member_id": integer, "page": 1, "page_size": 20 }`

Databases: MySQL `applications`.

`POST /applications/updateStatus`

Purpose: Update an application's status (recruiter workflow action).

Request Body: `{ "application_id": integer, "status": string }`

Validation rules: Status must be one of the five valid values.

Kafka event: `application.statusChanged` with `{old_status, new_status}` payload.

Databases: MySQL `applications` (UPDATE status), Kafka.

`POST /applications/addNote`

Purpose: Append a recruiter note to an application. Notes accumulate separated by `---`.

Request Body: `{ "application_id": integer, "note": string }`

Databases: MySQL `applications` (UPDATE recruiter_notes).

4.5 Messaging Service

Route prefixes: `/threads/*` and `/messages/*` (no single prefix — see individual routes)

`POST /threads/open`

Purpose: Create a new message thread between participants.

Request Body:

Field	Type	Required	Description
`participant_ids`	list[object]	Yes	Each: `{"user_id": int, "user_type": "member"|"recruiter"}`
`subject`	string	No	Thread subject line

Response: `MessageResponse` — thread object with `thread_id` and participant list.

Databases: MySQL `threads` (INSERT), MySQL `thread_participants` (INSERT for each participant).

POST /threads/get

Purpose: Retrieve thread metadata and participant list. Includes the most recent message.
Request Body: `{ "thread_id": integer }`
Databases: MySQL `threads`, `thread_participants`, `messages`.

POST /threads/byUser

Purpose: List all threads that a user participates in. Includes last message preview.
Request Body:

Field	Type	Required
`user_id`	int	Yes
`user_type`	string	Yes — `"member"` or `"recruiter"`
`page`	int	No — default 1
`page_size`	int	No — default 20

Databases: MySQL `thread_participants`, `threads`, `messages`.

POST /messages/send

Purpose: Send a message within a thread.
Request Body:

Field	Type	Required
`thread_id`	int	Yes
`sender_id`	int	Yes
`sender_type`	string	Yes — `"member"` or `"recruiter"`
`message_text`	string	Yes

Response: `MessageResponse` — message object with generated `message_id` and timestamp.
Validation rules: Sender must be a registered participant of the thread.
Failure handling: 3-retry loop with `db.rollback()` on each failure. After all retries exhausted, returns `success: false, message: "Message send failed. Please retry."`. Zero messages are written to the database on total failure.
Kafka event: `message.sent` with `{message_id, sender_type}` payload.
Databases: MySQL `messages` (INSERT with retry), Kafka.

POST /messages/list

Purpose: List messages in a thread, newest first.
Request Body: `{ "thread_id": integer, "page": 1, "page_size": 20 }`
Note: Client should reverse the list for chronological (chat-style) display.
Databases: MySQL `messages`.

4.6 Connection Service — `/connections`

Base URL prefix: `/connections`
Connection statuses: `pending`, `accepted`, `rejected`

POST /connections/request

Purpose: Send a connection request from one member to another.
Request Body: `{ "requester_id": integer, "receiver_id": integer }`
Validation rules: - Cannot connect with oneself. - Both member IDs must exist. - If a pending connection already exists in either direction: `success: false`. - If accepted: `success: false, message: "Already connected"`. - If previously rejected: re-request is allowed (reuses the existing row, updates to `pending`).
Kafka event: `connection.requested`.
Databases: MySQL `connections` (INSERT or UPDATE), Kafka.

POST /connections/accept

Purpose: Accept a pending connection request.
Request Body: `{ "connection_id": integer }`
Validation rules: Connection must be in `pending` status.
Side effects: Increments `connections_count` on both the requester and receiver member rows within the same transaction.
Kafka event: `connection.accepted`.
Databases: MySQL `connections` (UPDATE status), MySQL `members` (UPDATE connections_count × 2), Kafka.

POST /connections/reject

Purpose: Reject a pending connection request.
Request Body: `{ "connection_id": integer }`
Validation rules: Connection must be in `pending` status.
Databases: MySQL `connections` (UPDATE status to `"rejected"`).

POST /connections/list

Purpose: List all accepted connections for a member, enriched with the connected member's name and headline.
Request Body: `{ "user_id": integer, "page": 1, "page_size": 20 }`
Note: Returns only `accepted` connections. Pending connections are not listed.
Databases: MySQL `connections`, MySQL `members` (join for name/headline enrichment).

POST /connections/mutual

Purpose: Find mutual connections between two members.
Request Body: `{ "user_id": integer, "other_id": integer }`
Response: List of member objects (id, name, headline) that both users are connected to.
Algorithm: Set intersection of each user's accepted connection IDs.
Databases: MySQL `connections`, MySQL `members`.

4.7 Analytics Service

Route prefixes: `/events`, `/analytics` (no single prefix — shared router)

POST /events/ingest

Purpose: Ingest a tracking event from the UI or any service. Writes to MongoDB and publishes to Kafka.
Request Body:

Field	Type	Required
`event_type`	string	Yes
`actor_id`	string	Yes
`entity_type`	string	No
`entity_id`	string	No
`payload`	object	No

Databases: MongoDB `event_logs` (INSERT), Kafka.

POST /analytics/jobs/top

Purpose: Get top N jobs by a chosen metric within a look-back window.
Request Body:

Field	Type	Default	Description
`metric`	string	`"applications"`	`applications`, `views`, or `saves`
`limit`	int	10	Number of jobs to return
`window_days`	int	365	Look-back window in days

Databases: MySQL `applications`, `job_postings`, `saved_jobs` (aggregation query per metric).

POST /analytics/funnel

Purpose: View-to-save-to-apply funnel for a specific job.

Request Body: `{ "job_id": integer, "window_days": 365 }`

Response: `{views, saves, applications, view_to_save_rate, save_to_apply_rate, view_to_apply_rate}`

Databases: MySQL `job_postings`, `saved_jobs`, `applications`.

POST /analytics/geo

Purpose: City/state distribution of applicants for a job.

Request Body: `{ "job_id": integer, "window_days": 365 }`

Response: List of `{city, state, count}` ordered by count descending.

Databases: MySQL `applications` JOIN `members`.

POST /analytics/member/dashboard

Purpose: Member-facing analytics: 30-day profile view history and application status breakdown.

Request Body: `{ "member_id": integer }`

Response: `{profile_views_30d: [{date, views}], application_status_breakdown: {status: count}, total_connections, total_views_30d, total_applications}`

Databases: MySQL `profile_views_daily`, `applications`, `members`.

POST /analytics/jobs/top-monthly

Purpose: Top N jobs by application count grouped by calendar month. (Brief requirement: "Top 10 job postings by applications per month.")

Request Body: Same as `/analytics/jobs/top` — `{metric, limit, window_days}`.

Response: List of `{month (YYYY-MM), job_id, title, location, count}`.

Databases: MySQL `applications` JOIN `job_postings` with `DATE_FORMAT` grouping.

POST /analytics/geo/monthly

Purpose: City-wise applications grouped by calendar month for a selected job. (Brief requirement: "City-wise applications per month for a selected job posting.")

Request Body: `{ "job_id": integer, "window_days": 365 }`

Response: List of `{month, city, state, count}`.

Databases: MySQL `applications` JOIN `members` with `DATE_FORMAT` grouping.

POST /analytics/jobs/least-applied

Purpose: Open jobs with the fewest applications. Uses `LEFT JOIN` to include jobs with zero applications. (Brief requirement: "Top 5 job postings with the fewest applications.")

Request Body: `{ "limit": 5, "window_days": 90 }`

Response: List of `{job_id, title, location, count}` ordered ascending by count.

Databases: MySQL `job_postings` LEFT JOIN `applications`.

POST /analytics/jobs/clicks

Purpose: Click (view) counts per job from event data. (Brief requirement: "Clicks per job posting from logs.")

Request Body: `{ "limit": 10, "window_days": 30 }`

Response: List of `{job_id, title, clicks}` ordered by clicks descending, enriched with job titles from MySQL.

Data source (pre-aggregated): Reads from `analytics_job_clicks_daily` MongoDB collection — one document per `(job_id, date)` maintained by the Kafka `job.viewed` consumer handler. The aggregation

pipeline sums daily counters rather than scanning raw event logs, making this O(days × jobs) instead of O(raw events).

Fallback: If the pre-aggregated collection is empty (fresh deployment), the endpoint automatically falls back to scanning `event_logs` and returns the same response shape.

Databases: MongoDB `analytics_job_clicks_daily` (primary), MongoDB `event_logs` (fallback), MySQL `job_postings` (title enrichment).

POST /analytics/saves/trend

Purpose: Saved-job count aggregated by day or week. (Brief requirement: "Number of saved jobs per day/week from logs.")

Request Body: `{ "window_days": 30, "granularity": "day" | "week" }`

Response: List of `{period (YYYY-MM-DD or YYYY-WNN), count}`.

Data source (pre-aggregated): Reads from `analytics_saves_daily` MongoDB collection — one document per calendar day maintained by the Kafka `job.saved` consumer handler (`handle_job_saved`). Each document stores `date`, `week` (ISO-8601), and `saves` counter. Weekly roll-ups are computed in-process by grouping the small set of daily docs.

Fallback: If the pre-aggregated collection is empty, falls back to MySQL `saved_jobs` GROUP BY.

Databases: MongoDB `analytics_saves_daily` (primary), MySQL `saved_jobs` (fallback).

4.8 AI Agent Service — `/ai`

Full workflow documentation is in Section 7. This section provides the endpoint contracts.

POST /ai/analyze-candidates

Purpose: Start the multi-step Hiring Assistant workflow for a job. Returns immediately with a `task_id`. Work runs asynchronously in the background.

Request Body:

Field	Type	Default	Constraint
`job_id`	int	required	Must exist in MySQL
`top_n`	int	5	1–50

Response: `{ "task_id": "uuid", "job_id": integer }`

Side effects: Creates a task document in MongoDB `agent_tasks`. Publishes to Kafka `ai.requests`.

Databases: MongoDB (INSERT), Kafka.

POST /ai/task-status

Purpose: Check the current status and progress of an AI task.

Request Body: `{ "task_id": "uuid" }`

Response: Full task document including status, current step, progress percentage, and step history.

Task states: `queued` → `running` → `awaiting_approval` → `approved` / `rejected`; also `failed`, `interrupted` (server restart during execution).

Databases: Redis in-memory cache (primary), MongoDB `agent_tasks` (fallback on cache miss, enabling post-restart queries).

POST /ai/approve

Purpose: Human-in-the-loop decision: approve or reject the AI-generated shortlist and outreach drafts.

Request Body:

Field	Type	Required
`task_id`	string	Yes
`approved`	boolean	Yes
`feedback`	string	No — optional recruiter notes

Validation rules: Task must be in `awaiting_approval` status. Approving an already-approved task returns an error.

Side effects: Updates task status to `approved` or `rejected` in MongoDB. Stores feedback text on the task document.

Databases: MongoDB `agent_tasks` (UPDATE).

POST /ai/parse-resume

Purpose: Standalone resume parsing skill. Extracts structured fields from raw resume text.

Request Body: `{ "resume_text": "..." }` — minimum 10 characters.

Response: `{ method: "ollama" | "regex_fallback", name, email, phone, skills[], years_of_experience, education[], summary, success }`

Fallback behavior: If Ollama is unavailable (timeout or not running), falls back to regex-based extraction. The `method` field tells the caller which path was used.

POST /ai/match

Purpose: Standalone job-candidate scoring skill.

Request Body:

```
{
  "job_data": { "skills_required": [], "location": "...", "seniority_level": "...", "
work_mode": "..." },
  "candidate_data": { "skills": [], "location_city": "...", "location_state": "..." }
}
```

Response: `{ overall_score: 0.0–1.0, recommendation: "Strong Match" | ..., breakdown: { skills: float, location: float, seniority: float } }`

Scoring weights: Skills overlap 50%, location match 20%, seniority match 30%.

POST /ai/tasks/list

Purpose: List all currently active and recent AI tasks from the in-memory cache.

Response: List of `{task_id, job_id, status, created_at}`.

Note: Shows only tasks in the current server process's memory. Use `/ai/task-status` with a specific `task_id` for MongoDB-backed queries.

GET /ai/queue-status

Purpose: Real-time observability of the AI workflow dispatcher — how many workflows are running, how many are queued, and how many concurrency slots are free.

Request: No body.

Response:

```
{
  "success": true,
  "message": "2/2 workflows active, 1 queued",
  "data": {
    "queued": 1,
    "active": 2,
    "max_concurrent": 2,
    "available_slots": 0
  }
}
```

Purpose in architecture: The dispatcher enforces `MAX_CONCURRENT_WORKFLOWS = 2` to prevent all workflows from simultaneously calling Ollama (single-threaded LLM). Excess tasks wait in an `asyncio.Queue`. This endpoint exposes the queue depth and slot occupancy for monitoring and demo observability.

WebSocket /ai/ws/{task_id}

Protocol: WebSocket (ws://)

Purpose: Stream real-time AI task progress updates to the UI without polling.

On connect: Server immediately sends the current task status (from MongoDB if not cached).

During workflow: As each step completes, `update_task_status()` pushes JSON status updates to all connected clients for that `task_id`.

Keep-alive: Client sends `"ping"`, server responds `"pong"`.

Disconnect: Server removes the connection from the registry. The task continues running.

4.9 Authentication Service — `/auth`

Base URL prefix: `/auth`

JWT-based authentication using HS256 tokens. A separate `user_credentials` table stores hashed passwords; member and recruiter records are unchanged. Dependencies `require_member` and `require_recruiter` protect write operations on the respective domains.

POST /auth/login

Purpose: Authenticate with email and password. Returns a signed JWT.

Request Body:

```
{ "email": "jane@example.com", "password": "secret" }
```

Response:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "user_type": "member",
  "user_id": 101,
  "email": "jane@example.com"
}
```

Validation rules: Returns 401 if credentials are invalid.

Databases: MySQL `user_credentials` (SELECT, bcrypt verify).

POST /auth/login-form

Purpose: OAuth2 password-flow login using `application/x-www-form-urlencoded`. Used by the Swagger UI **Authorize** button.

Request Fields: `username` (email), `password` (form fields).

Response: Same shape as `/auth/login`.

POST /auth/register/member

Purpose: Create a new member profile and a linked credential record in a single transaction. Returns a JWT so the user is immediately authenticated.

Request Body:

Field	Type	Required
`email`	string	Yes — unique
`password`	string	Yes — min 8 chars
`first_name`, `last_name`	string	Yes
`headline`	string	No

Response: JWT token response (same as login).

Side effects: Inserts into MySQL `members` + `user_credentials`. Duplicate email returns `success: false`.

POST /auth/register/recruiter

Purpose: Create a new recruiter profile and linked credentials. Returns a JWT.

Request Body:

Field	Type	Required
`email`	string	Yes — unique
`password`	string	Yes — min 8 chars
`first_name`, `last_name`	string	Yes
`company_name`	string	No

Field	Type	Required
company_industry	string	No

Response: JWT token response.

Side effects: Inserts into MySQL `recruiters` + `user_credentials`.

GET /auth/me

Purpose: Return the current user's identity and profile object from the JWT token.

Auth required: `Authorization: Bearer <token>` header.

Response:

```
{
  "user_type": "member",
  "user_id": 101,
  "email": "jane@example.com",
  "profile": { "first_name": "Jane", "last_name": "Smith", ... }
}
```

Databases: MySQL `members` or `recruiters` (profile lookup by user_id).

Protected Endpoints

The following endpoint groups require a valid JWT with the matching `user_type`:

Guard	Applies to	Enforcement
require_member	POST /applications/submit	member_id in request must equal token's user_id
require_member	POST /connections/request	requester_id must equal token's user_id
require_member	POST /connections/accept, reject	Connection's receiver_id must equal token's user_id
require_member	POST /messages/send	sender_id must equal token's user_id
require_member	POST /members/update, delete	member_id must equal token's user_id
require_recruiter	POST /jobs/create	Recruiter identity verified via token
require_recruiter	POST /jobs/close	Job's recruiter_id must equal token's user_id
require_member	POST /jobs/save	member_id must equal token's user_id

Any protected endpoint called without a valid token returns `HTTP 401 Unauthorized`.

4.10 System / Health

GET /

Purpose: Root health check. Returns service name and version.

Response: `{ service, version, status: "running", docs: "/docs" }`

GET /health

Purpose: Detailed health check with per-service status.

Response:

```
{
  "status": "healthy" | "degraded",
  "services": {
    "api": true,
    "redis": true | false,
    "kafka_producer": true | false,
    "mongodb": true | false
  }
}
```

`"degraded"` is returned when any service is unhealthy. MySQL is not separately probed — if it is down, API operations will fail with 500s.

---

5. Kafka / Event Integration

Event Envelope Structure

Every Kafka message we publish follows the same JSON structure, enforced in `kafka_producer.py`:

```
{
  "event_type":      "job.viewed",
  "trace_id":        "550e8400-e29b-41d4-a716-446655440000",
  "timestamp":       "2026-04-05T10:00:00.000Z",
  "actor_id":        "42",
  "entity": {
    "entity_type":   "job",
    "entity_id":     "7"
  },
  "payload": {
    "domain_specific": "fields"
  },
  "idempotency_key": "a3f1e2b7-..."
}
```

Field	Notes
`event_type`	Matches the Kafka topic name (or is derivable from it)
`trace_id`	Same UUID across all steps of a multi-step workflow (AI workflow uses `task_id` as `trace_id`)
`timestamp`	ISO 8601 UTC
`actor_id`	The user or system component that triggered the event
`idempotency_key`	Fresh UUID per message; used by consumer for dedup

Topics Used

Topic	Producer endpoint	Consumer action
`job.created`	`POST /jobs/create`	Logs to MongoDB
`job.viewed`	`POST /jobs/get`	Increments `views_count` in MySQL; upserts into `analytics_job_clicks_daily` (MongoDB)
`job.saved`	`POST /jobs/save`	Logs to MongoDB; upserts into `analytics_saves_daily` (MongoDB)
`job.closed`	`POST /jobs/close`	Logs to MongoDB
`application.submitted`	`POST /applications/submit`	Increments `applicants_count` in MySQL
`application.statusChanged`	`POST /applications/updateStatus`	Logs to MongoDB
`message.sent`	`POST /messages/send`	Logs to MongoDB
`connection.requested`	`POST /connections/request`	Logs to MongoDB
`connection.accepted`	`POST /connections/accept`	Logs to MongoDB
`ai.requests`	`POST /ai/analyze-candidates`	AI workflow logs to MongoDB
`ai.results`	AI workflow steps (hiring_assistant.py)	AI result logs to MongoDB

Custom events from `POST /events/ingest` are published to `events.{event_type}` (dot replaced with underscore in topic name).

Consumer Group

A single consumer group `"linkedin-backend"` subscribes to the 11 topics listed above. The consumer is started as a background asyncio task during FastAPI lifespan startup. Note: `profile.viewed` is registered as a consumer topic in the codebase but is not currently published by any REST endpoint (see Section 10 limitations).

Delivery Guarantee — At-Least-Once with Manual Commit

We configured the consumer with `enable_auto_commit=False`. Offsets are committed manually only **after** a message has been fully processed (handler executed + idempotency record written to MongoDB). If the handler raises an exception, we intentionally do not commit the offset, so Kafka redelivers the message on the next consumer start.

This gives us **at-least-once delivery**: no message is silently lost, but a crashed handler may be called more than once. The idempotency layer converts this into effectively-exactly-once execution.

Commit sequence per message:

```
1. Poll message from Kafka
2. Idempotency check (in-memory set)
3. Idempotency check (MongoDB processed_events)
4. Execute handler
5. Write idempotency record to MongoDB   ← persisted BEFORE commit
6. consumer.commit()                     ← offset advanced AFTER persistence
```

Commit failures are logged as warnings (not re-raised); the worst case is re-delivery on the next start, handled by the idempotency layer.

Idempotency

We implemented two-layer deduplication in the consumer, applied before calling any handler:

1. **In-memory set** (`processed_keys`) — fast path, per process lifetime.
2. **MongoDB `processed_events` collection** — durable across restarts. Indexed on `idempotency_key` (unique index).

A message is skipped if its `idempotency_key` is found in either layer, guaranteeing exactly-once handler execution despite Kafka's at-least-once delivery.

Distributed Processing Support

---

6. Database Interaction Summary

MySQL Tables and Accessing Services

Table	Created by	Read by	Notes
`members`	`/members/create`	`/members/get`, `/members/search`, `/connections/list`, analytics	Cascades delete to applications, connections
`recruiters`	`/recruiters/create`	`/recruiters/get`, `/jobs/create`	Cascades delete to job_postings
`job_postings`	`/jobs/create`	`/jobs/get`, `/jobs/search`, `/jobs/byRecruiter`, analytics	Counts updated via Kafka consumer
`applications`	`/applications/submit`	`/applications/get`, `/applications/byJob`, `/applications/byMember`, analytics	Status updated via `/updateStatus`
`threads`	`/threads/open`	`/threads/get`, `/threads/byUser`	
`thread_participants`	`/threads/open`	`/threads/byUser`, `/messages/send` (participant check)	
`messages`	`/messages/send`	`/messages/list`	
`connections`	`/connections/request`	`/connections/list`, `/connections/mutual`	Unique constraint (requester, receiver)
`saved_jobs`	`/jobs/save`	`/analytics/saves/trend`, funnel	Unique constraint (member, job)
`profile_views_daily`	`seed_data.py`	`/analytics/member/dashboard`	One row per (member, date)

MongoDB Collections

Collection	Written by	Read by	Purpose
`event_logs`	`/events/ingest`, Kafka consumer handlers	`/analytics/jobs/clicks` (aggregation)	Append-only event log
`agent_tasks`	`hiring_assistant.start_task()`, `update_task_status()`	`/ai/task-status`, `rehydrate_tasks()`	AI task state (source of truth)
`agent_traces`	`run_hiring_workflow()` per candidate per step	Debugging/observability	Per-step AI trace records
`processed_events`	Kafka consumer (before handler execution)	Kafka consumer (dedup check)	Idempotency store

Redis Cache Keys

Key pattern	Set by	Evicted by	TTL
`members:get:{member_id}`	`/members/get`	`/members/update`, `/members/delete`	300s
`members:search:{keyword}:{skill}:{location}:{sort_by}:{cursor}`	`/members/search`	Any member write	60s
`recruiters:get:{recruiter_id}`	`/recruiters/get`	`/recruiters/update`, `/recruiters/delete`	300s
`jobs:get:{job_id}`	`/jobs/get`	`/jobs/update`, `/jobs/close`	300s
`jobs:search:{keyword}:{location}:{type}:{mode}:{seniority}:{sort_by}:{cursor}`	`/jobs/search`	Any job write	60s

7. AI API Workflow

Overview

We implemented the AI service using a **supervisor agent pattern**: the Hiring Assistant orchestrates three skills in a sequential pipeline, persisting state at each step, with a mandatory human approval gate before finalizing.

Dispatcher Queue and Concurrency Control

If the server restarts while tasks are `"queued"`, `rehydrate_tasks()` re-submits them to the queue. Tasks in `"running"` state at restart are marked `"interrupted"`.

Multi-Step Workflow

Task Lifecycle States

State	Meaning
`queued`	Task document created; workflow not yet started
`running`	Workflow is actively executing steps
`awaiting_approval`	Workflow complete; waiting for recruiter decision
`approved`	Recruiter approved the shortlist and outreach drafts
`rejected`	Recruiter rejected the output
`failed`	Unrecoverable error during execution
`interrupted`	Server restarted while task was `running` (mid-flight); `queued` tasks are re-submitted automatically

WebSocket Updates

We added a WebSocket at `ws://localhost:8000/ai/ws/{task_id}` to push real-time progress to the UI:

1. On connect, the server immediately sends the current task status (fetched from MongoDB if not in memory).
2. As each step completes, `update_task_status()` pushes a JSON update to all connected clients.
3. The client sends `"ping"` to keep the connection alive; the server responds `"pong"`.
4. WebSocket connections are process-scoped and do not survive server restarts.

Approval Flow

After `awaiting_approval`:

```
POST /ai/approve { task_id, approved: true, feedback: "" }
    → MongoDB: status = "approved", approval_feedback = feedback
    → Response: { success: true, message: "Task approved" }

POST /ai/approve { task_id, approved: false, feedback: "Shortlist not relevant" }
    → MongoDB: status = "rejected", approval_feedback = feedback
    → Response: { success: true, message: "Task rejected" }
```

Calling approve on a task not in `awaiting_approval` returns `success: false`.

Persistence and Restart Recovery

We write all task state to MongoDB as the primary store. On server restart, `rehydrate_tasks()` runs during FastAPI lifespan startup and performs three actions:

1. `awaiting_approval` tasks are loaded into the in-memory `active_tasks` dict — the recruiter approval endpoint continues to work.
2. `queued` tasks (never dispatched before the restart) are re-submitted to `_task_queue` and run normally once the dispatcher starts.
3. `running` tasks (mid-flight at restart time) are marked `interrupted` in MongoDB — these cannot be resumed and must be re-triggered via a new `/ai/analyze-candidates` call.

Skill	Ollama path	Fallback
Resume Parser	LLM prompt → structured JSON extraction	Regex patterns for email, phone, skills keywords
Job Matcher	Pure algorithm — no LLM involved	Always available
Outreach Generator	LLM prompt → personalized email	Fill-in template with candidate name and job title

10. API Readiness Summary

What Is Implemented

- **56 REST endpoints and 1 WebSocket endpoint** across all service domains: Profile, Recruiter, Job, Application, Messaging, Connection, Analytics, Authentication, and AI Agent.
- **Kafka integration**: 11 topics with a standard JSON envelope, single consumer group, and two-layer idempotency (in-memory + MongoDB).
- **Redis caching** with write-time invalidation. Cache-hit latency is 5–40× faster than cold MySQL on individual lookups (`cache_benchmark.py`).
- **Agentic AI hiring workflow**: supervisor pattern, three coordinated skills (resume parser, job matcher, outreach generator), HITL approval gate, MongoDB-persisted state, and dispatcher-queue concurrency control.
- **7 analytics endpoints** addressing the project brief's required charts (5 recruiter dashboard + 2 additional).
- **Failure mode handling** for duplicate entities, closed-job guard, message retry, Kafka idempotency, and AI restart recovery — each covered by integration tests.

What Is Strong

- Every endpoint is backed by a Pydantic v2 schema — contracts are machine-readable and auto-documented via Swagger at `/docs`.
- AI task state is fully durable: tasks survive server restarts. `queued` tasks are re-dispatched; `running` tasks are marked `interrupted` rather than silently lost.
- Kafka events carry a `trace_id` that links all steps of a multi-step workflow, enabling end-to-end observability.
- Cache invalidation is applied on every write operation that could affect a cached result.
- Application-level guards (duplicate check, closed-job check, message retry loop) leave the system in a consistent state after partial failures.

What Is Limited

- **No job `industry` search filter**: the `job_postings` schema does not have an industry column. Existing filters (keyword, location, type, mode, seniority, skills, salary) cover the demonstrated use cases.
- **`profile.viewed` Kafka event** is not published from `POST /members/get`, although the consumer handler exists. Profile view data in analytics is populated by the seeder.
- **Ollama inference is CPU-only** in the current deployment: expect 30–60 s per resume parse on local hardware. All endpoints remain functional via fallbacks when Ollama is unavailable.
- **Frontend CRUD coverage is partial**: member update/delete, recruiter CRUD, and job create/close are accessible via Swagger and the Postman collection but have no corresponding frontend UI form.

What Is Accessible via API but Not in the Frontend

The following are functional through Swagger (`/docs`) and the Postman collection, but do not have a corresponding UI form yet:

- Member update, delete
- Recruiter create, update, delete, get

- Job create, update, close, save
- Application status update, add note, list by job
- Full AI analyze → approve workflow (the AI Tools tab provides a partial UI)
- Analytics endpoints beyond the main dashboard charts

The Swagger UI at `http://localhost:8000/docs` provides a fully functional interactive interface for all 56 REST endpoints.